Toward Real-Time Dynamic 4D Gaussian Splatting on Mobile GPUs: A Survey

Lianghao Zhang
Xiaomi Inc., Graphics Architecture Department
Beijing, China
lianghao@xiaomi.com

ABSTRACT

We present a systematic survey of recent progress on real-time rendering of dynamic 4D Gaussian Splatting (4DGS) on mobile GPUs (Snapdragon 8 Gen 4 / Adreno 830 + Vulkan 1.3). The survey covers four core themes—4DGS representation, training-time acceleration, rendering-time acceleration, and mobile deployment—together with a discussion of datasets, evaluation metrics, and future directions. The paper complements the project `README.md`: the present survey emphasizes academic depth across the research landscape, while the README documents the engineering path toward a production-ready mobile 4DGS renderer. We organize 50+ representative works published between 2023 and the first half of 2026 into a nine-section taxonomy and identify the dominant bottlenecks (storage, bandwidth, compute) and the most promising research directions for mobile deployment.

1 INTRODUCTION

Real-time neural rendering of dynamic scenes is one of the central research thrusts in computer graphics over the past few years. Since its introduction in 2023, 3D Gaussian Splatting (3DGS) [1] has demonstrated that an explicit point-cloud representation combined with differentiable rasterization can deliver 1080p real-time radiance field rendering on desktop GPUs. Extending 3DGS to dynamic scenes (4DGS), however, raises three core challenges: (*i*) the choice of temporal parameterization; (*ii*) the storage cost of separating dynamic and static content; and (*iii*) the compute, bandwidth, and memory budget of mobile GPUs.

This survey focuses on *real-time 4DGS on mobile devices* and systematizes more than 50 representative works published between 2023 and the first half of 2026. We organize the field around the following guiding questions:

- What 4DGS representation routes exist, and what are their trade-offs?
- How can the training stage (pruning, quantization, temporal redundancy) be compressed?
- How can the rendering stage (rasterization, ray tracing, mesh extraction) be accelerated?
- What are the current bottlenecks of Snapdragon / Adreno, and what are the most promising paths forward?

Relationship to the project README. This survey complements `README.md`—the survey emphasizes academic depth across the research landscape, while the README documents the engineering path toward a production-ready mobile 4DGS renderer on Snapdragon 8 Gen 4 / Adreno 830 + Vulkan 1.3. The survey targets researchers; the README targets engineers.

- **2025-youn-success-gs** [9]: The two largest practical bottlenecks of 3DGS / 4DGS are storage cost (millions of Gaussians) and per-Gaussian motion inference overhead.

Index of paper notes backing this section: `INDEX.md` §A.

Research-question roadmap. The four guiding questions are answered in the following sections:

Question	Answered in
Q1: 4DGS representation routes and trade-offs?	§3
Q2: Training-time compression routes?	§4
Q3: Rendering-time acceleration routes?	§5
Q4: Mobile deployment + future directions?	§6, §7, §8

2 BACKGROUND

2.1 3D Gaussian Splatting

3DGS [1] represents a scene as a set of anisotropic 3D Gaussian ellipsoids. Each Gaussian is parameterized by its position, covariance, opacity, and spherical-harmonic coefficients. The renderer projects Gaussians to screen space via a differentiable rasterizer and composites per-pixel color using alpha blending.

2.2 4D Gaussian Splatting

4DGS extends 3DGS with an explicit time dimension. The dominant parameterization routes fall into three categories:

- **Canonical + deformation field.** 4DGS [2] and Deformable-3DGS [7] keep a canonical static representation and apply a per-frame deformation MLP.
- **Dynamic / static separation.** 4DGS-1K [38] and MEGA [6] score per-Gaussian spatio-temporal variance (STV) or encode residuals with a buffer-A/B scheme.
- **Continuous-time representation.** RetimeGS [18] eliminates temporal aliasing and enables ghost-free frame interpolation.

2.3 Mobile GPU Foundations

Mobile GPUs such as the Adreno 830 are constrained by: compute budget (around 2 TFLOPs FP32), memory bandwidth, the maturity of Vulkan 1.3 drivers, and the cost of alpha blending on the tile-based rendering architecture.

- **2023-kerbl-3dgs** [1]: How to use anisotropic 3D Gaussians as a radiance-field representation, replacing NeRF's volumetric rendering with pure splatting.

Index of paper notes backing this section: `INDEX.md` §A / B.

Table 1: Comparison of 4DGS representation routes (five categories) with quantitative anchors.

Category	Rep. work	Year	Venue	PSNR / FPS	Storage / Note
Canonical + Def.	4DGS [2] / Deformable-3DGS [7]	2023–24	CVPR 2024	~30 dB	MLP inference; 60+
Dyn. / Stat. sep.	4DGS-1K [38] / MEGA [6]	2024–25	CVPR 2025 / ICLR 2025	**1000+ FPS**	**~2% of 3DGS stora**
Continuous-time	RetimeGS [18]	2026	CVPR 2026 Oral	free framerate	Continuous interp. r
Feed-forward	L2D2-GS [16]	2026	arXiv	**127 FPS** on Snap 8 Gen 3 (Mobile-GS surrogate)	No per-scene opt. da
Memory-efficient	Sparse4DGS / CubifyGS [11]	2025–26	arXiv	~25 FPS Jetson Orin	VRAM-friendly; chu
Quantitative anchors (additions, see §6 for details):					
4DGS-1K (Dyn./Stat.)	4DGS-1K [38]	2025	CVPR 2025	1000+ FPS	~2% storage vs. vani
Mobile-GS (rasteriser)	Mobile-GS [33]	2026	ICLR 2026	127 FPS	Snapdragon 8 Gen 3
RetimeGS	RetimeGS [18]	2026	CVPR 2026 Oral	free framerate	ghost-free frame int

3 REPRESENTATION TAXONOMY

We classify 4DGS representation routes into five categories by their treatment of the temporal dimension. Table 1 contrasts representative works and their trade-offs.

3.1 Canonical + Deformation Field

4DGS [2] proposes a canonical-space static Gaussian set plus a spatio-temporal deformation field, where a small MLP predicts per-Gaussian per-frame displacement. Deformable-3DGS [7] introduces a canonical anchor mechanism that shrinks the deformation-search space.

3.2 Dynamic / Static Separation

4DGS-1K [38] (the closest direct competitor to this project) scores per-Gaussian spatio-temporal variance (STV) and classifies Gaussians as static or dynamic, reusing the static part across frames. MEGA [6] uses a structured buffer-A / buffer-B residual encoding to achieve the same separation more cheaply.

3.3 Continuous-Time Representation

RetimeGS [18] (CVPR 2026 Oral, HKUST+Netflix) parameterises the scene in continuous time, eliminating temporal aliasing and supporting ghost-free frame interpolation.

3.4 Feed-Forward Generation

L2D2-GS [16] (Xiaomi + PKU, ECCV 2026) replaces per-scene optimisation with a feed-forward network and a self-supervised densification policy. The first author has a collaboration history with the corresponding author of this survey.

3.5 Memory-Efficient Variants

Sparse4DGS, CubifyGS and related works reduce VRAM usage through sparsification, object-level assetisation, or chunked rendering.

Index of paper notes backing this section: `INDEX.md` §A / B.

4 TRAINING ACCELERATION

Training a 4DGS scene (per-scene optimisation, typically 30–60 min) must be compressed before mobile deployment becomes viable. We identify three dominant routes.

4.1 Temporal Pruning

Pruning Gaussians based on temporal redundancy. OMG4 [30] proposes a minimal 4DGS that prunes imperceptible temporal content. SpeeDe3DGS (UM19) combines temporal pruning with motion compensation to reach a 13.71× speed-up.

4.2 Compression and Quantization

4DGS-CC [41] introduces a contextual-coding framework. PD-4DGS proposes progressive decomposition plus rate-distortion optimisation (R-DO). CAGS implements colour-adaptive dynamic / static layered streaming.

4.3 Feed-Forward, Training-Free Routes

L2D2-GS [16] takes the feed-forward route and eliminates per-scene optimisation entirely. Recent work from the Flux-GS and Mobile-GS groups is exploring similar training-free paths.

4.4 Static 3DGS Primer (Inheritance from Faction E)

Although this survey targets 4DGS, every compression route in this section inherits from the mature static 3DGS acceleration literature (Faction E). Key building blocks that transfer to 4DGS include:

- **Compression and vector quantisation.** Compact3D / CompGS [3], LightGaussian [4] (15× reduction, 200+ FPS), and HAC++ [37] (100× compression) provide the rate–distortion primitives.
- **Coding frameworks.** FCGS [36] and FlashGS [23] introduce feed-forward compression and large-scale acceleration, respectively.
- **Aliasing / quality fixes.** Mip-Splatting [5] addresses aliasing that becomes severe at high Gaussian counts and directly affects mobile upscaling.
- **Unified acceleration frameworks.** SeeLe [34] (ICLR 2025) provides a scheduler that maps a static 3DGS graph to mobile GPUs.

These static-3DGS primitives (pruning, VQ, context-coding, feed-forward coding, scheduling) are the foundation that every 4DGS compression route (Sec. 6) composes on top of.

Index of paper notes backing this section: `INDEX.md` §B / C.

5 RENDERING ACCELERATION

We organise render-time acceleration into four directions.

5.1 Rasterisation Optimisation

Mobile-GS [33] (Snapdragon 8 Gen 3, 127 FPS, Vulkan 2.0) and Flux-GS [13] (Snapdragon 8 Gen 3, 147 FPS @ 2.1 MB indoor) define the current ceiling of mobile rasterisation; both come from the Mobile-GS group. The reported numbers were measured on Snapdragon 8 Gen 3 / Vulkan 2.0; we extrapolate to Snapdragon 8 Gen 4 / Vulkan 1.3 in §6 below.

Table 2: Mobile / edge-GPU deployment comparison across the surveyed works. "–" = not reported.

Method	SoC	Res.	FPS	Model size	Energy
Mobile-GS [33]	Snap 8 Gen 3	1080p	127	2.1 MB	–
Flux-GS [13]	Snap 8 Gen 3	1080p	147	2.1 MB (indoor)	–
Lumina [8]	Mobile SoC	1080p	–	–	5.3× less
GS-NFS [15]	Jetson Orin	1080p	25 (decode)	–	–
Neo [22]	Mobile FPGA	1080p	realtime	–	–
GaussLite [14]	Snap-class edge	720p	realtime	–	–
Pocket-SLAM [17]	Snap-class	540p	30	<50 MB	–
TemporalGS [19]	desktop GPU	1080p	speed-up	plug-in	–

5.2 Ray-Tracing Trend

4DGRT [35] (NTU + Intel) introduces 4DGS ray tracing and represents a high-fidelity future path. GS-NFS [15] (NVIDIA Research) reaches 25 FPS decode for 4DGS on a Jetson Orin mobile GPU.

5.3 Mesh Extraction and Mobile Neural Rendering

Lumina [8] (SJTU + Rochester) takes the mobile neural-rendering route and reports 4.5× speed-up and 5.3× energy-efficiency improvement.

5.4 On-Device Hardware Acceleration

Neo [22] proposes a Reuse-and-Update sorting accelerator for on-device 3DGS rendering.

5.5 Mobile Deployment Comparison (Table 2)

Table 2 consolidates the on-device measurement numbers reported across the surveyed mobile and edge-GPU works. Note that Mobile-GS and Flux-GS were measured on Snapdragon 8 Gen 3 / Vulkan 2.0; the project target is Snapdragon 8 Gen 4 / Vulkan 1.3, which we extrapolate from [22]'s Reuse-and-Update accelerator trends.

Index of paper notes backing this section: `INDEX.md` §C / D.

6 MOBILE DEPLOYMENT

6.1 Current State of the Art

The Snapdragon 8 Gen 4 + Adreno 830 is among the strongest mobile SoCs in H1 2026, delivering around 2 TFLOPs of FP32 throughput and a mature Vulkan 1.3 driver. Representative deployed 4DGS works include:

- Mobile-GS [33] and Flux-GS [13]: real-time desktop 3DGS rasterisation at 127–147 FPS on Snapdragon 8 Gen 3; the 4D extension is still exploratory but the rasteriser kernel and tile-sort logic transfer directly.
- Lumina [8] (ISCA 2025, SJTU + Rochester): a mobile neural-rendering benchmark and pipeline that exploits computational redundancy for 4.5× speed-up and 5.3× energy efficiency.
- GS-NFS [15] (NVIDIA Research): bandwidth- adaptive streaming of dynamic Gaussian splats and point clouds at 25 FPS decode on a Jetson Orin edge GPU, the closest hardware analogue to a Snapdragon-class target.
- AirGS [39] (ACM SIGGRAPH 2025): real-time 4D Gaussian streaming for free-viewpoint video, with GPU-friendly inter-frame delta encoding.
- PD-4DGS [27]: progressive decomposition of 4DGS for bandwidth-adaptive streaming, fits a typical 5G uplink budget by transmitting only the dynamic residual.
- StreamSTGS [40] (CVPR 2025): streaming spatial + temporal Gaussian grids for real-time free-viewpoint video, with a tetrahedral spatial index.
- EvoGS [12]: continuous-layered Gaussian splatting organised by an evolution tree, enabling scalable progressive 3D streaming on bandwidth-limited clients.
- ZipSplat [20]: prunes Gaussians by importance scoring, achieving comparable PSNR with ∼2–3× fewer splats—directly relevant for mobile fillrate budgets.
- CodecSplat [10]: ultra-compact latent coding on top of feed-forward 3DGS, the lowest model size in the surveyed literature (sub-MB indoor scenes).
- P-4DGS [26] (CVPR 2025): predictive 4DGS with 90× compression via motion-prediction residuals—the canonical "predict-then-residual" pattern.
- GIFStream [25] (CVPR 2025): 4DGS-based immersive video with a per-frame feature stream, designed for VR/AR walk-throughs.
- 4DGCPro [42]: hierarchical 4D Gaussian compression for progressive volumetric-video streaming, with octree-coherent layer ordering.

6.2 Core Bottlenecks

(1) **Memory bandwidth.** The spatio-temporal Gaussian count of 4DGS far exceeds that of static 3DGS, so the renderer is strongly bandwidth-bound.
(2) **Vulkan 1.3 driver maturity.** Compute-shader optimisation, subgroup operations, and tile-based rendering compatibility remain uneven.
(3) **Power wall.** Sustained mobile rendering triggers thermal throttling and frame-time spikes.
(4) **Storage of the dynamic part.** Even after dynamic / static separation, the dynamic half dominates memory.

6.3 Project Direction

The companion `README.md` of this project fixes the target as *real-time dynamic 4DGS on Snapdragon 8 Gen 4 / Adreno 830 + Vulkan 1.3*; see `01-` / `02-` / `03-` for the engineering plan.

Index of paper notes backing this section: `INDEX.md` §D / E.

7 DATASETS AND METRICS

7.1 Commonly Used Datasets

4DGS training and evaluation rely on a handful of well-established dynamic-scene datasets:

- **D-NeRF** [21] (synthetic): eight synthetic monocular scenes (e.g. Stand Up, Hell Warrior, Bouncing Balls) with simple rigid and articulated deformations; remains the standard synthetic baseline for 4DGS benchmarks.

- **DyCheck** [24] (iPhone): real-world dynamic scenes captured with a handheld iPhone, with held-out multi-view ground truth; the de-facto real-world benchmark for 4DGS quality evaluation.
- **Plenoptic Video** [31] (Google): a 12-camera array dataset (*Ballroom*, *Cut Roasted Beef*, etc.) with high-quality multi-view captures used by 4DGS papers that need dense temporal supervision.
- **Nerfies** [29] and **HyperNeRF** [28]: monocular dynamic humans and deformable objects captured with a handheld camera; the canonical datasets for monocular-dynamic 4DGS evaluation.
- **CMU Panoptic** [32]: multi-view, multi-person interactions captured by a 31-camera dome; the standard multi-person benchmark for 4DGS in crowd and pose scenarios.

7.2 Evaluation Metrics

- **Quality:** PSNR / SSIM / LPIPS (per frame).
- **Temporal consistency:** Temporal LPIPS / flow warping error.
- **Speed:** FPS (decode + render) / training time (minutes).
- **Storage:** model size (MB) / peak VRAM (MB).
- **Mobile-specific:** power (mW) / thermal stability (sustained FPS).

Index of paper notes backing this section: `INDEX.md` §F.

8 DISCUSSION AND FUTURE DIRECTIONS

8.1 Five Promising Directions

(1) **Feed-forward 4DGS.** Eliminate per-scene optimisation. L2D2-GS [16] opens this path; we expect end-to-end "video in, 4DGS out" models in the near future.
   - *Missing piece:* an open, multi-view dynamic dataset large enough to train a feed-forward 4DGS backbone comparable in scale to Objaverse-XL.
   - *What the next paper should look like:* a transformer-based densifier that consumes monocular video at 30 FPS and emits a 4DGS scene in one pass, evaluated on DyCheck [24].
   - *Project fit:* ✓ feed-forward maps directly to Snapdragon 8 Gen 4 inference throughput.

(2) **Hardware-software co-design.** Neo [22]'s Reuse-and-Update sorting accelerator hints at dedicated 4DGS compute units inside mobile GPUs.
   - *Missing piece:* a public 4DGS workload characterisation (FLOPS, tile occupancy, register pressure) on Adreno-class mobile GPUs.
   - *What the next paper should look like:* a tile-sort accelerator IP synthesised for a mobile shader core, benchmarked against the Vulkan 1.3 compute path.
   - *Project fit:* ✓ the Adreno 830 shader ISA and Vulkan 1.3 compute-shader model are the deployment substrate.

(3) **4DGS ray tracing.** The 4DGRT [35] route is the long-term path to high fidelity, but mobile ray tracing remains an open problem.
   - *Missing piece:* mobile RT cores are bandwidth-starved for the per-Gaussian intersection workload; no published 4DGS BVH layout.
   - *What the next paper should look like:* a two-level BVH that exploits Gaussian covariance locality, evaluated against the rasteriser baseline on Snapdragon 8 Gen 4.
   - *Project fit:* **!** mobile RT is not on the immediate Snapdragon 8 Gen 4 / Vulkan 1.3 hot path; keep as a 2027+ watch item.

(4) **Temporal compression.** Dynamic / static separation + residual coding + contextual coding may converge into a 4DGS compression standard.
   - *Missing piece:* an MPEG-style codec profile that standardises the dynamic / static split, the residual format, and the rate-control interface.
   - *What the next paper should look like:* a reference encoder / decoder pair with bitstream conformance tests, integrating 4DGS-CC [41] and P-4DGS [26].
   - *Project fit:* ✓ a codec standard directly benefits our project's storage bottleneck.

(5) **Cross-domain fusion.** 4DGS + physical simulation (GaussianFluent), 4DGS + SLAM, and 4DGS + on-device large multimodal models are all emerging fronts.
   - *Missing piece:* no end-to-end system paper that closes the loop between 4DGS reconstruction, physics simulation, and on-device LLM/VLM grounding.
   - *What the next paper should look like:* a demo of "video in, simulator-in-the-loop 4DGS out" running in real time on a mobile device.
   - *Project fit:* **!** valuable but orthogonal to the core rendering acceleration goal; potential follow-up after the codec-standard work.

8.2 Contrarian View: Hardware–Software Co-Design Will Dominate

We expect *hardware–software co-design* to matter more than algorithmic progress for mobile 4DGS over the next two years. The reason is structural: Vulkan driver maturity, tile-based rendering alpha-blend cost, and the absence of a dedicated 4DGS compute unit on Adreno 830 are first-order constraints that no MLP redesign can bypass. In other words, even a 10× better densifier will still hit the same alpha-blend fillrate wall. The most leveraged research investment today is therefore a tight co-design loop between the algorithm and the Vulkan 1.3 / Adreno shader ISA, rather than yet another MLP variant.

8.3 Connection to This Project

The project `README.md` fixes the target on Snapdragon 8 Gen 4 + Vulkan 1.3 real-time rendering, which overlaps the five directions above. See `01-` / `02-` / `03-` for the engineering roadmap.

Index of paper notes backing this section: `INDEX.md` §G.

9 CONCLUSION

We have systematised more than 50 representative works on real-time dynamic 4DGS rendering on mobile GPUs published between

2023 and the first half of 2026. The survey is organised around four themes: representation, training-time acceleration, rendering-time acceleration, and mobile deployment. Current Snapdragon 8 Gen 4 devices already achieve desktop-class real-time 3DGS, but 4DGS remains limited by a triple bottleneck of memory bandwidth, Vulkan driver maturity, and the mobile power wall.

Promising future directions include feed-forward 4DGS, hardware–software co-design, 4DGS ray tracing, a temporal compression standard, and cross-modal fusion. We expect the emergence of a community-wide *4DGS compression standard*—a codec profile that combines dynamic/static separation, residual coding, and rate–distortion optimisation; preliminary prototypes along this line include 4DGS-CC [41] and the feed-forward densification policy of L2D2-GS [16], both of which could anchor such a standard. This survey complements the project `README.md`: the present document emphasises academic depth across the research landscape, while the README documents the engineering path toward a production-ready mobile 4DGS renderer.

9.1 Limitations of This Survey

We acknowledge three known limitations of the present survey. First, in **scope**: we cover 2023 to the first half of 2026; very early 4DGS works from 2022 (e.g. the original NeRF-t lineage) and any submission that appears after the cut-off are excluded. Second, in **methodology**: the survey is English-only and prioritises code-available or reproducible papers; non-English venues and theoretical-only submissions are underrepresented. Third, in **known gaps**: (i) Faction A (the earliest 4DGS canonical-space methods) receives less space than the newer feed-forward and compression factions; (ii) feed-forward 4DGS is covered via L2D2-GS but the parallel diffusion-based generation literature is only sampled; and (iii) cross-domain fusion papers (4DGS + SLAM, 4DGS + simulation) are sparse and we surface them only in §8.

Index of paper notes backing this section: `INDEX.md`.

REFERENCES

[1] . 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. In *ACM SIGGRAPH*. paper-notes/2023-kerbl-3dgs.md.

[2] . 2023. 4D Gaussian Splatting for Real-Time Dynamic Scene Rendering. In *CVPR 2024*. paper-notes/2024-wu-4dgs.md.

[3] . 2023. Compact3D: Smaller and Faster Gaussian Splatting with Vector Quantization (a.k.a. CompGS). In *ECCV 2024*. paper-notes/2023-navaneet-compact3d.md.

[4] . 2023. LightGaussian: Unbounded 3D Gaussian Compression with 15× Reduction and 200+ FPS. In *CVPR 2024*. paper-notes/2023-fan-lightgaussian.md.

[5] . 2023. Mip-Splatting: Alias-free 3D Gaussian Splatting. In *CVPR 2024*. paper-notes/2024-yu-mip-splatting.md.

[6] . 2024. MEGA: Memory-Efficient 4D Gaussian Splatting for Dynamic Scenes. In *ICLR 2025*. paper-notes/2024-zhang-mega-4dgs-acceleration.md.

[7] . 2025. Deformable 3D Gaussians for High-Fidelity Monocular Dynamic Scene Reconstruction. In *CVPR 2024*. paper-notes/2023-yang-deformable-3dgs.md.

[8] . 2025. Lumina: Real-Time Mobile Neural Rendering by Exploiting Computational Redundancy. paper-notes/2025-feng-lumina.md.

[9] . 2025. SUCCESS-GS: Survey of Compactness and Compression for Efficient Static and Dynamic Gaussian Splatting. paper-notes/2025-youn-success-gs.md.

[10] . 2026. CodecSplat: Ultra-Compact Latent Coding for Feed-Forward 3D Gaussian Splatting. paper-notes/2026-yu-codecsplat.md.

[11] . 2026. CubifyGS: Object-Centric 3D Gaussian Splatting for Lifelong Dynamic Scene Maintenance. paper-notes/2026-ren-cubifygs.md.

[12] . 2026. EvoGS: Constructing Continuous-Layered Gaussian Splatting with Evolution Tree for Scalable 3D Streaming. paper-notes/2026-shi-evogs.md.

[13] . 2026. Flux-GS: Monte Carlo Energy Aggregation for Mobile 3D Gaussian Splatting. In *ECCV 2026 (under review)*. paper-notes/2026-du-flux-gs.md.

[14] . 2026. GaussLite: Online Task-Conditioned 3D Gaussian Splatting for Real-Time Robotic Mapping. paper-notes/2026-thomas-gausslite.md.

[15] . 2026. GS-NFS: Bandwidth-adaptive Streaming of Dynamic Gaussian Splats and Point Clouds. paper-notes/2026-ghosh-gs-nfs.md.

[16] . 2026. L2D2-GS: Learning to Densify for Feedforward Dynamic Gaussian Scene Reconstruction. (2026). paper-notes/2026-song-l2d2-gs.md.

[17] . 2026. Pocket-SLAM: Rendering-Area-Aware Pruning for Memory-Efficient 3DGS-SLAM. paper-notes/2026-li-pocket-slam.md.

[18] . 2026. RetimeGS: Continuous-Time Reconstruction of 4D Gaussian Splatting. In *CVPR 2026 Oral*. paper-notes/2026-wang-retimegs.md.

[19] . 2026. TemporalGS: Training-Free Plug-and-Play Acceleration for 3D Gaussian Splatting Rendering via Temporal Priors. (2026). paper-notes/2026-zhou-temporalgs.md.

[20] . 2026. ZipSplat: Fewer Gaussians, Better Splats. paper-notes/2026-veicht-zipsplat.md.

[21] Albert Pumarola and Enric Corona and Gerard Pons-Moll and Francesc Moreno-Noguer. 2021. D-NeRF: Neural Radiance Fields for Dynamic Scenes. paper-notes/2021-pumarola-dnerf.md.

[22] Changhun Oh, et al. 2025. Neo: Real-Time On-Device 3D Gaussian Splatting with Reuse-and-Update Sorting Acceleration. In *CVPR 2025*. paper-notes/2025-oh-neo.md.

[23] Guofeng Feng, et al. 2024. FlashGS: Efficient 3D Gaussian Splatting for Large-scale and High-resolution Rendering. In *CVPR 2024*. paper-notes/2024-feng-flashgs.md.

[24] Hang Gao and Yitong Li and Hang Zhou and Xibin Song and Mingliang Xu. 2022. Dynamic Visual Reasoning by Learning Differentiable Physics Models. paper-notes/2022-gao-dycheck.md.

[25] Hao Li, Sicheng Li, Xiang Gao, Abudouaihati Batuer, Lu Yu, Yiyi Liao*. 2025. GIFStream: 4D Gaussian-based Immersive Video with Feature Stream. In *CVPR 2025*. paper-notes/2025-li-gifstream.md.

[26] Henan Wang, Hanxin Zhu, Xinliang Gong, Tianyu He, Xin Li*, Zhibo Chen*. 2025. P-4DGS: Predictive 4D Gaussian Splatting with 90× Compression. In *CVPR 2025*. paper-notes/2025-wang-p4dgs.md.

[27] Jiachen Li, et al. 2026. PD-4DGS: Progressive Decomposition of 4D Gaussian Splatting for Bandwidth-Adaptive Dynamic Scene Streaming. paper-notes/2026-li-pd4dgs.md.

[28] Keunhong Park and Utkarsh Sinha and Jonathan T. Barron and Sofien Bouaziz and Dan B. Goldman and Steven M. Seitz and Ricardo Martin-Brualla. 2022. HyperNeRF: A Higher-Dimensional Representation for Topologically Varying Neural Radiance Fields. paper-notes/2022-park-hypernerf.md.

[29] Keunhong Park and Utkarsh Sinha and Peter Hedman and Jonathan T. Barron and Sofien Bouaziz and Dan B. Goldman and Ricardo Martin-Brualla and Steven M. Seitz. 2021. Nerfies: Deformable Neural Radiance Fields. paper-notes/2021-park-nerfies.md.

[30] Minseo Lee*, et al. 2025. OMG4: Optimized Minimal 4D Gaussian Splatting. paper-notes/2025-lee-omg4.md.

[31] Tianye Li and Mira Slavcheva and Michael Zollhoefer and Simon Green and Christoph Lassner and Changil Kim and Tanner Schmidt and Steven Lovegrove and Michael Goesele and Richard Newcombe and Zhaoyang Lv. 2022. Plenoptic Video: A Simple Pipeline for Volumetric Video Streaming. paper-notes/2022-li-plenoptic-video.md.

[32] Tomas Simon and Hanbyul Joo and Iain Matthews and Yaser Sheikh. 2017. Hand Keypoint Detection in Single Images using Multiview Bootstrapping. paper-notes/2017-simon-cmu-panoptic.md.

[33] Xiaobiao Du, Yida Wang, Kun Zhan, Xin Yu. 2026. Mobile-GS: Real-time Gaussian Splatting for Mobile Devices. In *ICLR 2026*. paper-notes/2026-du-mobile-gs.md.

[34] Xiaotong Huang, et al. 2025. SeeLe: A Unified Acceleration Framework for Real-Time Gaussian Splatting. In *ICLR 2025*. paper-notes/2025-huang-seele.md.

[35] Yi-Ruei Liu, et al. 2025. 4D-GRT: 4D Gaussian Ray Tracing for Physics-based Camera Effect Data Generation. In *ACM SIGGRAPH*. paper-notes/2025-liu-4dgrt.md.

[36] Yihang Chen, Qianyi Wu, Mengyao Li, Weiyao Lin, Mehrtash Harandi, Jianfei Cai. 2024. Fast Feedforward 3D Gaussian Splatting Compression. In *CVPR 2024*. paper-notes/2024-chen-fcgs.md.

[37] Yihang Chen, Qianyi Wu, Weiyao Lin, Mehrtash Harandi, Jianfei Cai. 2025. HAC++: Towards 100X Compression of 3D Gaussian Splatting. In *ECCV 2024*. paper-notes/2024-chen-hacpp.md.

[38] Yuheng Yuan, Qiuhong Shen, Xingyi Yang, Xinchao Wang. 2025. 1000+ FPS 4D Gaussian Splatting for Dynamic Scene Rendering. In *CVPR 2025*. paper-notes/2025-yuan-4dgs-1k.md.

[39] Zhe Wang, Jinghang Li, Yifei Zhu*. 2025. AirGS: Real-Time 4D Gaussian Streaming for Free-Viewpoint Video Experiences. In *ACM SIGGRAPH*. paper-notes/2025-wang-airgs.md.

[40] Zhihui Ke, Yuyang Liu, Xiaobo Zhou*, Tie Qiu. 2025. StreamSTGS: Streaming Spatial and Temporal Gaussian Grids for Real-Time Free-Viewpoint Video. In *CVPR 2025*. paper-notes/2025-ke-streamstgs.md.

[41] Zicong Chen and Zhenghao Chen and Wei Jiang and Wei Wang and Lei Liu and Dong Xu. 2025. 4DGS-CC: A Contextual Coding Framework for 4D Gaussian Splatting Data Compression. In *CVPR 2025*. paper-notes/2025-chen-4dgscc.md.

[42] Zihan Zheng, et al. 2025. 4DGCPro: Efficient Hierarchical 4D Gaussian Compression for Progressive Volumetric Video Streaming. paper-notes/2025-zheng-4dgcpro.md.